

Mixed-Integer Programming Formulation

Unrelated Parallel Machine Scheduling with Sequence-Dependent Setup Times

$R \mid s_{ij} \mid C_{\max}$

Josa Ferreira*

June 8, 2026

Abstract

This portfolio project presents a Mixed-Integer Linear Programming (MILP) model for the unrelated parallel machine scheduling problem with sequence-dependent setup times ($R \mid s_{ij} \mid C_{\max}$). The formulation minimizes the total makespan ($C_{\max}$) for independent jobs across heterogeneous machines. Featuring a complete mathematical framework including assignment, flow conservation, and subtour elimination constraints, this project serves as a technical demonstration of applying mathematical optimization to complex manufacturing and logistics challenges.

Contents

1	Mathematical Formulation	1
1.1	Problem Description	1
1.2	Sets	1
1.3	Parameters	2
1.4	Decision Variables	2
1.5	Model	2
2	Computational Results	2

Keywords: Parallel Machine Scheduling Problem; Makespan; Sequence Dependent Setup.

1 Mathematical Formulation

1.1 Problem Description

The problem $R \mid s_{ij} \mid C_{\max}$ consists of scheduling a set of independent jobs on a set of unrelated parallel machines. Under this configuration, each job must be processed exactly once on any machine. Because the machines are unrelated, they process jobs at varying speeds. Furthermore, transitioning between consecutive jobs on the same machine incurs a sequence-dependent setup time. The objective is to minimize the makespan, defined as the time at which the last job completes processing.

1.2 Sets

- $M = \{1, 2, \dots, m\}$: Set of m unrelated parallel machines.
- $N = \{1, 2, \dots, n\}$: Set of n available jobs.
- $N_0 = N \cup \{0\}$: Extended set of jobs, including a dummy source node (0) representing the start of a machine's sequence.

*josenildo.junior9321@gmail.com

1.3 Parameters

- p_{ij} : Processing time of job $j \in N$ on machine $i \in M$.
- s_{ijk} : Sequence-dependent setup time incurred when transitioning from job $j \in N_0$ to job $k \in N_0$ on machine $i \in M$.

1.4 Decision Variables

- $x_{ijk} = 1$ if job $j \in N_0$ immediately precedes job $k \in N_0$ on machine $i \in M$; 0 otherwise.
- $y_{ij} = 1$ if job $j \in N$ is processed on machine $i \in M$; 0 otherwise.
- $C_{\max}$: Continuous variable representing the overall makespan.

1.5 Model

The mixed-integer linear programming (MILP) model is formulated as follows:

$$\min \quad C_{\max} \quad (1)$$

$$\text{S.t.} \quad \sum_{i \in M} y_{ij} = 1 \quad \forall j \in N \quad (2)$$

$$\sum_{\substack{k \in N_0 \\ k \neq j}} x_{ijk} = y_{ij} \quad \forall i \in M, j \in N \quad (3)$$

$$\sum_{\substack{j \in N_0 \\ j \neq k}} x_{ijk} = \sum_{j \in N_0, j \neq k} x_{ikj} \quad \forall i \in M, k \in N \quad (4)$$

$$\sum_{k \in N_0} x_{i0k} = 1 \quad \forall i \in M \quad (5)$$

$$\sum_{j \in S} \sum_{k \in N_0 \setminus S} x_{ijk} \geq y_{ih}, \quad \forall h \in S, S \subset N_0, i \in M \quad (6)$$

$$\sum_{j \in N_0} \sum_{\substack{k \in N_0 \\ k \neq j}} s_{ijk} x_{ijk} + \sum_{j \in N} p_{ij} y_{ij} \leq C_{\max}, \quad \forall i \in M \quad (7)$$

The objective function (1) minimizes the makespan ($C_{\max}$). Constraints (2) ensure that each job is assigned to and processed by exactly one machine. Constraints (3) link the routing and assignment variables, dictating that if a job is processed on a machine, it must immediately precede another node in that machine's sequence. Constraints (4) enforce flow conservation, guaranteeing that a machine must leave any job node it enters. Constraints (5) require every machine to initiate its routing sequence from the dummy origin (node 0). Constraints (6) represent the subtour elimination constraints (SECs), which prevent invalid isolated loops within the job sequences. Subtour elimination constraints were implemented as lazy constraints, given that their total count grows exponentially with the number of jobs. Finally, Constraints (7) defines makespan.

2 Computational Results

Following the benchmark protocol proposed by [1], 20 instances were generated: 10 with $m = 2$ machines and $n = 5$ jobs, and 10 with $m = 4$ machines and $n = 10$ jobs. Processing times (p) and setup times (s) were independently sampled from the discrete uniform distribution $UD(1, 99)$. To ensure reproducibility, all generated instances and their corresponding results were stored in a SQLite database (instances.sqlite). Table 1 presents the optimal makespan ($C_{\max}$) and solver runtime (in seconds) obtained for the 20 randomly generated instances under the two problem configurations.

The results illustrate the expected growth in computational effort as the instance size increases. For $m = 2, n = 5$, the solver consistently finds the optimum in under 0.2 seconds (average ≈ 0.097 s). Scaling to $m = 4, n = 10$ raises the average runtime to ≈ 11.6 s, with the hardest instance (index 8) requiring 30.3 seconds.

Table 1: Computational results for small and medium instances.

m	n	Instance	$C_{\max}$	Time (s)
2	5	0	184.0	0.098
2	5	1	168.0	0.118
2	5	2	178.0	0.022
2	5	3	217.0	0.044
2	5	4	206.0	0.188
2	5	5	214.0	0.138
2	5	6	164.0	0.098
2	5	7	207.0	0.040
2	5	8	170.0	0.042
2	5	9	218.0	0.179
4	10	0	168.0	10.060
4	10	1	157.0	5.048
4	10	2	142.0	8.521
4	10	3	161.0	8.679
4	10	4	147.0	11.886
4	10	5	179.0	6.523
4	10	6	180.0	16.772
4	10	7	151.0	6.149
4	10	8	153.0	30.336
4	10	9	150.0	12.305

References

- [1] Eva Vallada and Rubén Ruiz. A genetic algorithm for the unrelated parallel machine scheduling problem with sequence dependent setup times. *European Journal of Operational Research*, 211(3):612–622, 2011.